Introduction to Python Programming

Overview of Sudoku Software Project Group 7 Anja Graf

Overview of most important classes

01

User control

user_interface.py

02

Game logic

game.py

03

Sudoku logic

sudoku.py

04

Player accounts

player.py

05

Saving & Loading

persistence.py

06

Further helper classes

colors.py & instructions.py

User Control – user_interface.py

- Call the main method of this class to start the program
- This class controls every input and output through console
- It also creates an instance of game, player and persistence class
- Most important methods: choose_input_options_pre_game() and choose_input_options_during_game()

```
if __name__ == "__main__":
    ui = UI()
```

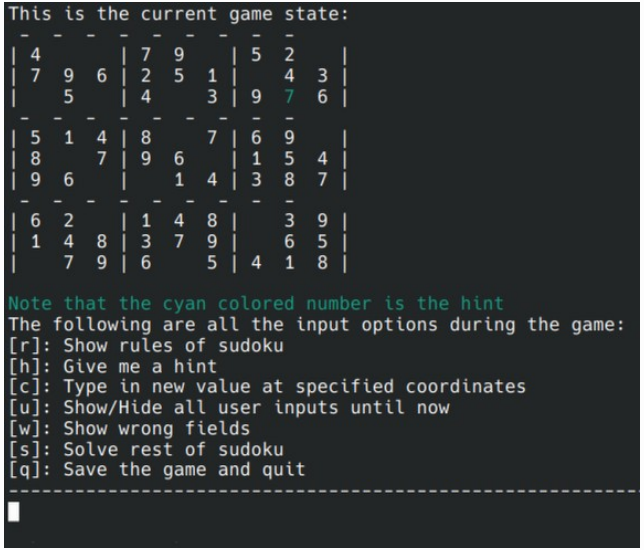
user_interface.py at line 573

```
Welcome to Sudoku!
The following are all the input options before the game:
[n]: Start a new sudoku game
[d]: Set the difficulty (easy, medium, hard). The current difficulty is EASY
[s]: Set size of sudoku. The current size is 9
[p]: Set player. The current player is not set
[o]: Open game from file
[t]: Type in own game
[h]: Show all high scores
[r]: Show rules of sudoku
--------------------------------------------------------------------------------
```

Prints out this pregame menu and reads in user input afterwards

Game Logic – game.py

- Takes the users input for the game size (default 9x9) and difficulty (Easy, Medium, Hard)
- With that parameters it generates a new sudoku field
- This class controls the game flow as well as the Sudoku field and functions as abstract wrapper
- To differentiate between the generated field and the current field including all user inputs it has two attributes, second one is a deep copy of the first
- It also has functionality like the timer to measure the needed time to solve the sudoku field or the counter for all hints the user needed



Menu during the game,
Showing hint functionality

Code example

This shows the code of how the solving animation is created.
Note that:

- The return value in the solve() method is a generator because of the yield keyword
- This way, printing the next step can be delayed by 0.2 seconds so the player can actually watch an animation. Else it would seem like the complete sudoku field was solved instantly

```
for step in self.game.solve():
    UI.clear()
    print("Solving current sudoku...")
    print(step)
    time.sleep(0.2)
```

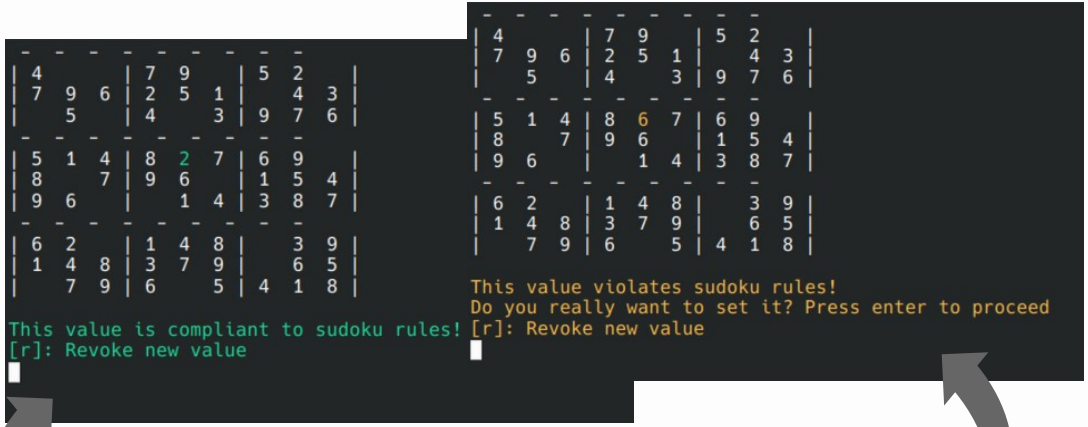
user_interface.py
at line 303

```
def solve(self) -> Generator:
    """
    This repeatedly applies the show_and_set_hint() method so the player can watch the sudoku solving itself.
    """
    while not self.sudoku.is_solved():
        self.hint_counter += 1
        yield self.sudoku.show_and_set_hint()
```

game.py
at line 141

Sudoku Logic – sudoku.py

- Here the sudoku field will be actually generated and gets represented via a two dimensional array
- Aside from that this class provides an automatic solving algorithm
- And, of course, it contains the checker for if the game is finished, thus all fields are filled correctly



Returns false

Returns true

```
def is_valid(self) -> bool:
    """Checks if all sudoku field entries are valid.
    return (
        self._is_valid_in_every_row()
        and self._is_valid_in_every_column()
        and self._is_valid_in_every_grid()
    )
```

sudoku.py at line 376

Code example

This shows the code to what was displayed on previous slide.
Note that:

- The is_valid() method of sudoku is used here
- user_interface.py uses the game class wrapper to access the sudoku field indirectly

user_interface.py at line 482

```
def _set_value(self, x, y, value) -> bool:
    """
    This is a helper method to set a new value 'value' at '(x,y)' coordinates in the current sudoku.
    It checks if the new value violates the sudoku rules and asks for confirmation of the user if it does.
    Nevertheless, the user can still choose to input the wrong value
    (for example, the other value causing the violation could be the actual wrong one).

    Also the user can revoke his input. Then the state of the game remains like before.
    """
    pre_value = self.game.sudoku.values[x][y]
    self.game.sudoku.set(x, y, value)
    UI.clear()
    if self.game.sudoku.is_valid():
        print(self.game.sudoku.show_and_set_value(
            x, y, value, Colors.GREEN))
        print(Colors.to_right_str("This value is compliant to sudoku rules! Press enter to proceed \
            \n[r]: Revoke new value"))
    else:
        print(self.game.sudoku.show_and_set_value(
            x, y, value, Colors.YELLOW))
        print(Colors.to_may_str("This value violates sudoku rules! \
            \nDo you really want to set it? Press enter to proceed \
            \n[r]: Revoke new value"))
    if input() == "r":
        self.game.sudoku.set(x, y, pre_value)
        print("The value was not set!")
        return False
    print("The value was set successfully!")
    return True
```

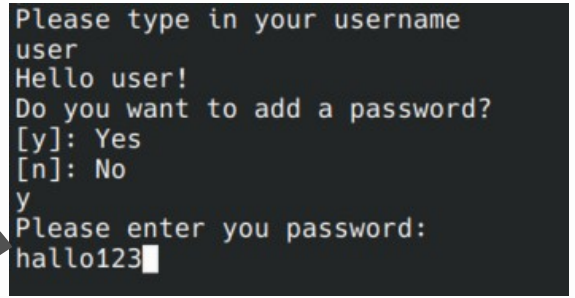
Player accounts – player.py

- This class manages all player accounts
- Each player can choose a name and password, which will then be hashed with sha 256 algorithm, but can also be left out

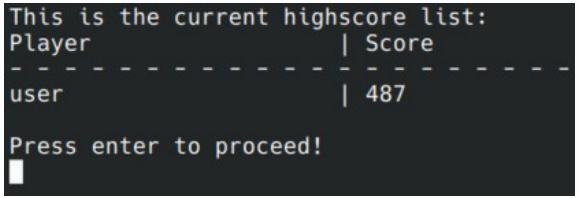
Hashed into



Example of user-data.json where all players are saved automatically with their attributes



Creating a new User



Calling the highscore list

Saving & Loading – persistence.py

- This class hold everything that should be permanently saved, such as e.g. the current game
- It also covers methods regarding the file system like checking if a string is a directory or a file path or gets the current working directory which is used as default one

```
Sudoku Game from 2025-02-28 18:14:59.391076

 -  -  -  -
| 1  4 | 2  3 |
| 2  3 | 1    |
 -  -  -  -
| 3    | 4    |
|    2 | 3  1 |

Needed time: 5 seconds
Needed hints: 0
```

Example textfile at /home/anja

```
[q]: Save the game and quit
[f]: Change the current filename [Sudoku Game 2025-02-28 18:14:59.391076]
[p]: Change the current filepath [/home/anja]
--------------------------------------------------------------------------------
q
The game [Sudoku Game 2025-02-28 18:14:59.391076, Difficulty.EASY, 2 x 2, Progress: 0/4] was successfully saved at [/home/anja].
Goodbye, thanks for playing sudoku!
```

Saving the current game with default name (timestamp) and default path

Further helper classes – colors.py & instructions.py

- There are further classes that structure the code and help to understand it better
- One of them is instructions.py, which only consists of explanations and options used in the user_interface class, that were too voluminous and are outsourced here

```
def _show_input_options_during_game():
    print("The following are all the input options during the game:\
        \n[r]: Show rules of sudoku \
        \n[h]: Give me a hint \
        \n[c]: Type in new value at specified coordinates \
        \n[u]: Show/Hide all user inputs until now\
        \n[w]: Show wrong fields \
        \n[s]: Solve rest of sudoku \
        \n[q]: Save the game and quit \
        \n--------------------------------------------------------------------------------")
```

Example method in instructions.py at line 19

- The second one is colors.py that collects different ansi escape sequences as enumeration. This is used while printing colored output in the console

Code example

This shows the code
of how a game is saved.
Note that:

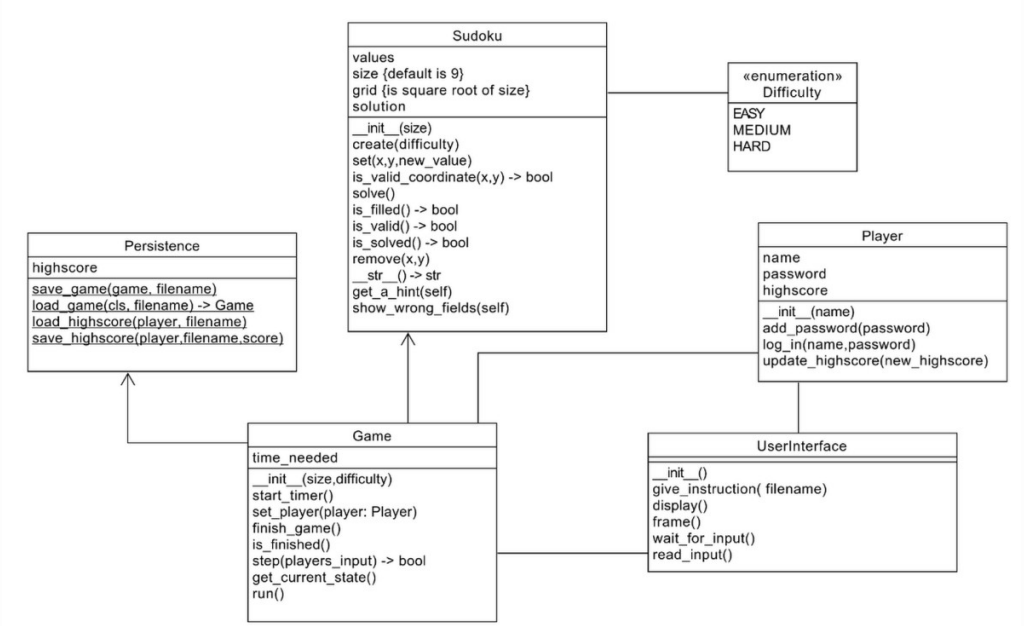
- The default name is the current time and the default path is the current working directory as the open statement gets an empty string as argument in that case
- The path module from os helps to use this code independently from the operating system
- The game class supports the __str__() method and returns a string that contains every needed information about the game.
  This way the code here is much more simplified as now it is possible to just call str(game)

```
    @staticmethod
    def save_game(game: Game, path: str=""):
        """Saves a game. ···
        if game.name is None:
            game.name = "Sudoku Game " + str(dt.datetime.now())
        with open(os.path.join(path,game.name),"w") as f:
            content = "Sudoku Game from " + str(dt.datetime.now())
            if game.ui.player is not None:
                content += ", played by " + game.ui.player.name
            content += str(game)
            f.write(content)
```

persistence.py at line 53

Thanks for your interest!

https://git.rz.uni-augsburg.de/coling-tutor/python-winter-2425-sopro-group7



Due to lack of space, only code examples were shown here. Feel free to check out the repo, as you now know the high level structure!